Phishing Detector Project Report

Date: June 25, 2026

1. Project Overview

The Phishing Detector project is a hybrid security application designed to identify phishing attempts in email content. It combines a Flask-based web interface, a Python-based machine learning detection module, and a Gmail Chrome extension for client-side monitoring.

2. Components

- Flask web application (`app.py`) for manual email content submission and analysis.
- Machine learning inference module (`model/detector.py`) that uses saved model artifacts to classify email text.
- Chrome extension (`chrome-extension/`) that inspects Gmail content and warns users about suspicious messages.
- Training utilities (`model/train.py`) and a synthetic dataset for model retraining.

3. Architecture and Workflow

The system uses a hybrid detection strategy. Incoming email text is first analyzed using rule-based heuristics, and if the ML model can load, the model also performs a prediction. Risk is calculated from heuristic indicators and machine learning confidence, producing a final risk score that is shown to the user.

4. Detection Rules

The application applies a set of phishing detection heuristics covering common malicious email patterns:

- Urgent or threatening language such as “verify”, “suspended”, “locked”, or “act now”.
- Suspicious URLs, including IP-based links, unusual top-level domains, excessive subdomains, and brand impersonation domains.
- Free email services used in a business or security context, such as gmail.com or yahoo.com for official correspondence.
- Generic greetings like “Dear customer” or “Hello user” that often appear in bulk phishing emails.
- Requests for sensitive data including passwords, credit card details, or social security numbers.
- Attachment and invoice language suggesting unauthorized file downloads or payment actions.

- Typosquatting and brand misspellings that mimic legitimate services.

5. Chrome Extension Behavior

The Gmail Chrome extension scans page content and applies the same phishing heuristics in the browser context. It detects malicious phrases, suspicious URLs, email addresses, generic greetings, and impersonation attempts, then displays a risk score badge and analysis details to the user.

6. Installation and Usage

Installation steps for the project are:

- Clone the repository and navigate to its root directory.
- Create and activate a Python virtual environment.
- Install dependencies from `requirements.txt`.
- Verify the model artifact files exist in `model/`.
- Run `python app.py` to start the Flask application and open the web interface.
- Load the `chrome-extension/` folder into Chrome via chrome://extensions with Developer mode enabled.

7. Project Structure

The project structure includes the Flask app root, model artifacts and training utilities, and the browser extension folder with manifest, scripts, and UI assets.

8. Limitations and Future Improvements

- The Chrome extension is targeted at Gmail only and may not work on other email providers without adaptation.
- The ML model relies on pre-trained artifacts; retraining or expanding data may improve accuracy.
- Rule-based heuristics can produce false positives for legitimate emails containing strong security language.
- The user interface and extension feedback could be enhanced with clearer warnings and remediation guidance.

9. Conclusions

This project provides a practical phishing detection system that combines heuristic scanning with machine learning. It is suitable for educational use, proof-of-concept security demos, and as a foundation for further email protection features.

End of Report